

Enunciado Proyecto 2 : Donaciones

SISTRA-TEC: Sistema de Trazabilidad de Donaciones

Objetivo: Las personas estudiantes deben desarrollar una aplicación web que "funcione", además de ser escalable, segura y bien arquitecturada. El equipo de desarrollo debe asegurar que el software sea inclusivo antes de lanzarlo a toda la comunidad.

Se recomienda que la persona estudiante lea el artículo que ofrece un método científico para identificar "facetas cognitivas" donde el software suele fallar a las usuarias, no por falta de capacidad, sino por sesgos de diseño.

- GenderMag: A Method for Evaluating Software's Gender Inclusiveness
- "Gender-Inclusivity Software" with Margaret Burnett (48.4 min)
<https://www.youtube.com/watch?v=txp4CI3JGbc>

Reflexión Crítica

"Si en este aula somos 32 hombres y 1 mujer, y diseñamos el software según como nosotros aprendemos... ¿Estamos construyendo herramientas que excluyen al 50% de la población mundial por puro sesgo cognitivo?"

Ejercicio: "SISTRA-TEC: Sistema de Trazabilidad de Donaciones"

1. El Problema

Tras una emergencia natural en Costa Rica, diversas organizaciones recogen donaciones, pero la ciudadanía a menudo desconfía del destino final de los bienes. Las personas estudiantes deben construir una Plataforma Web de Trazabilidad en Tiempo Real que permita a un donante ver el ciclo de vida de su ayuda (desde la recepción hasta la entrega al beneficiario).

2. Requerimientos Técnicos

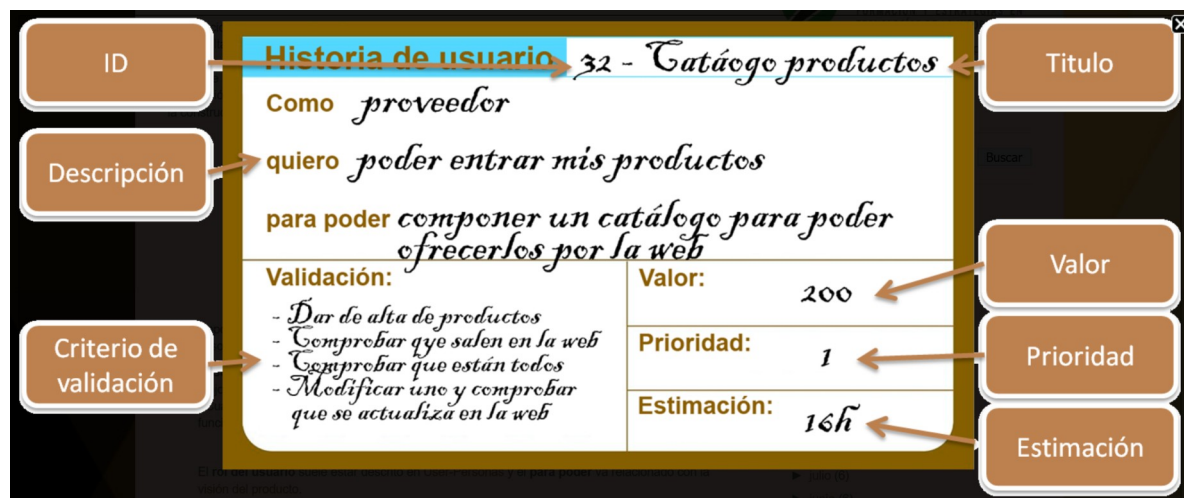
El sitio a construir no debe ser un sitio estático. Debe ser una MVA (Minimum Viable Architecture):

- **Frontend:** React, Angular o Vue. Uso de gestión de estado global (Redux/Context API) y consumo de APIs asíncronas.
- **Backend:** Node.js (Express), Python (FastAPI/Django) o C# (.NET Core). Implementación de arquitectura en capas o limpia (Clean Architecture).
- **Base de Datos:** PostgreSQL para la integridad de las transacciones de inventario.
- **Autenticación:** Implementación de JWT (JSON Web Tokens) , OAuth2 para diferenciar roles (Administrador de Centro, Transportista, Donante).

3. Fases del Ejercicio

Fase A: Modelado y Contratos

Definir cuatro Historias de Usuario adicionales a la HU "Registro de Donación". Utilizar el formato:



Fase B: Desarrollo del Core

- **Frontend Inclusivo:** Basado en las cinco facetas cognitivas del artículo de Burnett, aplicar los principios de *GenderMag* y accesibilidad (WCAG) en los

formularios de registro para asegurar que cualquier ciudadano pueda usarlo.

- **Lógica de Negocio:** Implementar el "Estado de la Donación" (Recibido -> Clasificado -> En Tránsito -> Entregado).

Fase C: Despliegue y Calidad

- **Muro de Género :** identificar al menos un "Muro de Género" en el software desarrollado.
- **Pruebas Unitarias:** Cobertura mínima del 70% en la lógica del backend.
- **Despliegue:** Uso de contenedores (Docker) o servicios en la nube (Heroku, AWS, Azure).



Herramienta de Evaluación: La "Métrica de Producción"

Criterio	Nivel 3: Senior (TEC)	Nivel 2: Junior	Nivel 1: Estudiante
Arquitectura	Separación total de intereses (Frontend/API/DB).	Lógica mezclada en los controladores.	Todo en un solo archivo de script.
Seguridad	Contraseñas hasheadas y rutas protegidas por JWT.	Rutas protegidas, pero contraseñas en texto plano.	Sin seguridad ni validación.
UX/Inclusión	Sigue la guía de lenguaje inclusivo y es responsive.	Funciona en desktop, pero no es intuitivo.	Solo funciona en la resolución del alumno.



Mini-Manual: Auditoría de Inclusión Cognitiva (GenderMag)

El método *GenderMag* ayuda a encontrar "Muros de Género" en el software evaluando cómo diferentes perfiles cognitivos interactúan con la interfaz. No se trata de "hacer software para mujeres", sino de hacer software para todos los estilos de pensamiento.

1. El Proceso de Auditoría (Paso a Paso)

Suponga que Abigail es una usuaria con baja autoeficacia en computación; prefiere métodos familiares y tiene aversión al riesgo (no "travesea" botones nuevos). Suponga además que Tim es un usuario con alta autoeficacia; disfruta explorar nuevas funciones y aprende por "ensayo y error".

Para cada acción que estos usuarios deban realizar en el software (ej: *Crear una cuenta*), hacer las preguntas críticas:

1. **Visión de la Faceta:** ¿La motivación (o riesgo/autoeficacia) de "Abigail" y de "Tim" les permitiría ver que esta acción es necesaria?
 2. **Visión de la Acción:** ¿"Abigail" y "Tim" sabrán *cómo* realizar la acción con los elementos que tiene en pantalla?
 3. **Visión del Progreso:** Si realiza la acción, ¿el software le da una respuesta que refuerce su confianza o la confunde más?
-

2. Checklist de Rediseño Inclusivo

Si se detecta un fallo, aplicar estas soluciones de ingeniería:

- [] **Mensajes de Error Constructivos:** En lugar de "Error 504", usar "No pudimos conectar, intenta de nuevo, tus datos están a salvo". (Protege la Autoeficacia).
 - [] **Botón de "Deshacer" (Undo) Visible:** Si el usuario sabe que puede volver atrás, se atreverá a explorar más. (Reduce el Riesgo).
 - [] **Explicación del "Por Qué":** Antes de pedir un dato, explica para qué sirve. (Ayuda al procesamiento Holístico).
 - [] **Ayuda en Contexto:** No mandes al usuario a un manual externo; pon pequeñas notas de ayuda junto a los campos difíciles. (Apoya el Aprendizaje).
-